Course: Parallel and Distributed Computing (ECOM 6339)

Student Name: Dalya Awad Mohammad Hassan

Student ID: 220253121

Assignment: Assignment 02

Date: 2 November 2026

Task 1: Parallel Matrix Multiplication

Experimental Environment

The experiments were conducted on an Apple MacBook Pro equipped with an Apple M4 Pro processor (12 CPU cores: 8 performance cores and 4 efficiency cores) and 24 GB of unified memory. GNU GCC 16.2.0 was used with OpenMP support and the -O2 optimization flag. Two double-precision matrices of size 2048 × 2048 were multiplied. Each configuration was evaluated using five timed executions following an untimed warm-up.

Compilation command: `gcc-16 -O2 -fopenmp matrix_mul.c -o matrix_mul -lm`

1. **Write a serial matrix multiplication for C = A × B with N = 2048 and double-precision elements.**

A serial matrix multiplication algorithm was implemented in C using three nested loops. For two square matrices A and B, each element of the result matrix C is calculated as:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} B_{kj}$$

The implementation uses double-precision floating-point values and has a computational complexity of $O(N^3)$.

Serial implementation:

```
void matmul_serial(const double *A, const double *B,
                   double *C, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++)
                sum += A[(size_t)i*n+k] * B[(size_t)k*n+j];
            C[(size_t)i*n+j] = sum;
        }
    }
}
```

The serial algorithm was executed five times. Its mean execution time was 21.197242 seconds, which was used as the baseline for calculating parallel speedup and efficiency.

2. Parallelize with #pragma omp parallel for.

The serial algorithm was parallelized by applying #pragma omp parallel for to the outermost loop. Each thread computes independent rows of the output matrix, while matrices A and B remain shared and read-only.

Parallel implementation:

```
void matmul_parallel(const double *A, const double *B,
                     double *C, int n, int threads) {
    #pragma omp parallel for num_threads(threads) schedule(static)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++)
                sum += A[(size_t)i*n+k] * B[(size_t)k*n+j];
            C[(size_t)i*n+j] = sum;
        }
    }
}
```

Static scheduling was selected because each output row requires approximately the same computational work. Since each thread writes to distinct output elements, the implementation avoids conflicting writes and does not require explicit synchronization inside the multiplication loops.

The OpenMP runtime successfully created the requested thread teams of 1, 2, 4, 8, and 16 threads.

3. Verify correctness against the serial version (maximum absolute error).

The parallel results were compared against the serial reference using the maximum absolute error:

$$E_{\max} = \max_{0 \le i,j < N} \left| C_s(i,j) - C_p(i,j) \right|$$

A tolerance of 10^{-9} was used to determine correctness.

Threads	Maximum absolute error	Status
1	0	PASS
2	0	PASS
4	0	PASS
8	0	PASS
16	0	PASS

All configurations produced a maximum absolute error of zero across the five repeated executions. Therefore, the parallel implementation produced results identical to the serial reference for the tested matrices.

4. Measure runtime for threads = 1, 2, 4, 8, 16.

Dalya Mohammad Hassan
220253121

Execution time was measured using the OpenMP wall-clock function omp_get_wtime(). Five measurements were collected for each thread configuration after an untimed warm-up.

The mean runtime was calculated using:

$$\overline{T}(p) = \frac{1}{5}\sum_{r=1}^{5} T_r\,(p)$$

Table 1. Measured execution times (seconds)

Configuration	Run 1	Run 2	Run 3	Run 4	Run 5	Mean
Serial	20.6544	21.8299	21.7201	20.2657	21.5162	21.1972
1 thread	21.4146	21.3408	22.0417	20.9679	21.5230	21.4576
2 threads	10.2638	10.2384	10.5687	11.0884	10.4998	10.5318
4 threads	7.1378	6.2634	7.3472	6.7492	7.0557	6.9106
8 threads	3.3171	3.2724	3.2430	3.3297	3.3053	3.2935
16 threads	3.2968	3.2978	3.3718	3.3465	3.3060	3.3238

Execution time decreased substantially as the number of threads increased from one to eight. The eight-thread configuration achieved the shortest mean runtime of 3.293501 seconds. Increasing the thread count to 16 produced a slightly higher mean runtime of 3.323797 seconds, indicating diminishing performance benefits.

5. Compute speedup and efficiency.

Speedup was calculated relative to the independent serial implementation:

$$S(p) = \frac{T_s}{T_p}$$

Parallel efficiency was calculated as:

$$E(p) = \frac{S(p)}{p}$$

For example, at eight threads:

$$S(8) = \frac{21.197242}{3.293501} = 6.4361$$

$$E(8) = \frac{6.436081}{8} \times 100 = 80.45\%$$

Table 2. Parallel performance results

Threads	Mean runtime (s)	Speedup	Efficiency (%)
1	21.457594	0.9879	98.79
2	10.531808	2.0127	100.63
4	6.910633	3.0673	76.68
8	3.293501	6.4361	80.45
16	3.323797	6.3774	39.86

The highest measured speedup was 6.4361, obtained using eight threads, with a parallel efficiency of 80.45%.

The efficiency at two threads slightly exceeded 100%, which may reflect measurement variability or differences in execution conditions. This observation alone is insufficient to establish sustained superlinear speedup.

At 16 threads, efficiency decreased to 39.86%, without any improvement in mean runtime compared with eight threads. Since the Apple M4 Pro contains 12 physical CPU cores, additional thread scheduling and resource contention may contribute to the limited scaling, although these effects were not directly measured.

Task 1 Conclusion

The OpenMP implementation successfully parallelized matrix multiplication while preserving numerical correctness. All tested configurations produced a maximum absolute error of zero.

The eight-thread configuration achieved the best measured performance, reducing mean execution time from 21.197242 seconds in the serial version to 3.293501 seconds, corresponding to a speedup of 6.4361 and parallel efficiency of 80.45%.

The results demonstrate that increasing parallelism can substantially improve computational performance, but adding more threads does not necessarily produce further acceleration. The experiment illustrates the importance of workload characteristics, processor architecture, and parallel efficiency when evaluating multicore performance.

Task 2: Scheduling Comparison

6. **For the same matrix multiply, compare schedule(static), schedule (dynamic, 1), schedule (dynamic, 16), and schedule(guided).**

Four OpenMP scheduling strategies were implemented and compared: static, dynamic(1), dynamic(16), and guided.

Static scheduling distributes iterations in advance with minimal scheduling overhead. Dynamic scheduling assigns chunks as threads become available, while guided scheduling progressively

reduces chunk sizes. All strategies were evaluated using eight OpenMP threads and the same matrix multiplication algorithm.

7. Record runtime for each at 8 threads.

Each configuration was executed five times after an untimed warm-up. Execution times were measured using omp_get_wtime(), and the arithmetic mean was calculated.

Table 3. Regular Matrix Multiplication Results

Scheduling strategy	Mean runtime (seconds)
Static	3.430517
Dynamic (1)	3.341226
Dynamic (16)	3.417193
Guided	3.270824

All configurations produced a maximum absolute error of zero.

8. Explain which schedule is best for this workload and why.

Guided scheduling achieved the lowest measured mean runtime of 3.270824 seconds, approximately 4.66% faster than static scheduling.

Although static scheduling is theoretically suitable for regular workloads because iteration costs are uniform, guided scheduling performed best in this experiment.

The observed difference may be influenced by runtime scheduling, system activity, or the heterogeneous processor architecture. Additional experiments would be required to identify the exact cause.

9. Now implement a triangular loop (work proportional to i) and repeat.

A triangular matrix multiplication workload was implemented by changing the inner output-column loop to j <= i. Consequently, the number of computed output elements increases with the row index, creating an irregular workload.

Table 4. Triangular Workload Results

Scheduling strategy	Mean runtime (seconds)
Static	2.650530
Dynamic (1)	1.607526
Dynamic (16)	1.714007
Guided	1.634143

All configurations passed correctness verification with a maximum absolute error of zero.

10. Explain how the optimal schedule differs.

The optimal measured scheduling strategy differed between the two workloads.

For regular matrix multiplication, guided scheduling achieved the fastest mean runtime of 3.270824 seconds. Since the computational cost per row is approximately uniform, static scheduling is also theoretically suitable.

For the triangular workload, dynamic scheduling with chunk size 1 achieved the fastest mean runtime of 1.607526 seconds, approximately 39.35% faster than static scheduling in terms of execution-time reduction.

The triangular workload has uneven computational costs because later iterations perform more work. Dynamic scheduling improves load balancing by allowing threads to request additional iterations as they finish.

These results demonstrate that the most effective OpenMP scheduling strategy depends on workload characteristics. Regular workloads generally benefit from low scheduling overhead, whereas irregular workloads can benefit from dynamic load balancing.

Task 2 Conclusion

The scheduling experiments demonstrated that workload distribution significantly influences OpenMP performance. Guided scheduling achieved the best measured performance for regular matrix multiplication, while Dynamic (1) performed best for the irregular triangular workload.

All implementations passed correctness verification. The results highlight the importance of selecting scheduling strategies according to workload characteristics rather than applying one scheduling policy universally.

Task 3: Race Condition and Its Fixes

Consider the following racy loop:

```
int counter = 0;

#pragma omp parallel for

for (int i = 0; i < 10000000; i++) {

counter++;

}

printf("counter = %d\n", counter);
```

11. Run the program multiple times and record the counter value.

A shared counter was incremented 10,000,000 times using eight OpenMP threads without synchronization. The program was executed five times.

Table 5. Unsynchronized Counter Results

Run	Counter value
1	2,612,665
2	1,254,520
3	1,255,926
4	1,277,138
5	1,263,362

None of the executions produced the expected value of 10,000,000, demonstrating incorrect behavior caused by concurrent unsynchronized updates.

12. Explain why the result differs from 10,000,000.

A data race occurs because multiple threads concurrently read and modify the same shared counter without synchronization. The counter++ operation involves reading, incrementing, and writing the variable. Overlapping operations may overwrite updates made by other threads.

Consequently, the program has undefined behavior and does not guarantee the expected counter value.

13. Fix it using #pragma omp critical and measure the time.

The counter update was protected using an OpenMP critical section:

```
#pragma omp critical
{
    counter++;
}
```

The critical section ensures that only one thread updates the counter at a time.

All five executions returned the correct value of 10,000,000.

Mean execution time: 0.143214 seconds.

14. Fix it using #pragma omp atomic and measure the time.

The counter increment was protected using an atomic update:

```
#pragma omp atomic update
counter++;
```

The atomic directive prevents concurrent updates from interfering with one another.

All five executions returned the correct value of 10,000,000.

Mean execution time: 0.168587 seconds.

15. Fix it using reduction(+:counter) and measure the time.

An OpenMP reduction was applied to the shared counter:

```
#pragma omp parallel for num_threads(8) reduction(+:counter)
for (int i = 0; i < 10000000; i++) {
   counter++;
}
```

The reduction clause creates private accumulation variables for the threads and combines their results at the end of the parallel loop.

All five executions returned the correct value of 10,000,000.

Mean execution time: 0.000051 seconds.

16. Compare the three fixes and explain which is fastest and why.

Table 6. Synchronization Performance

Method	Mean execution time (s)	Correct result
Critical	0.143214	PASS
Atomic	0.168587	PASS
Reduction	0.000051	PASS

The reduction method achieved the shortest measured execution time, followed by critical and atomic synchronization.

Critical sections serialize access to the shared counter. Atomic operations protect individual updates but still introduce contention when multiple threads increment the same memory location. Reduction minimizes contention by performing accumulation in private variables.

In this experiment, critical was approximately 15% faster than atomic. This result reflects the measured execution environment and should not be generalized to all workloads.

The exceptionally low reduction runtime may also reflect compiler optimization of the simple increment-only computation. Therefore, the measured difference should not be interpreted solely as synchronization overhead.

The reduction method achieved the shortest measured execution time (0.000051 seconds), followed by critical (0.143214 seconds) and atomic (0.168587 seconds). Although reduction minimizes

contention through thread-private accumulation, its exceptionally short runtime may also reflect compiler optimization of the simple increment loop. Therefore, the measured performance difference should not be attributed entirely to synchronization overhead.

Task 3 Conclusion

The experiment demonstrated that unsynchronized concurrent updates could produce incorrect results. All three synchronization approaches corrected the counter value.

Reduction achieved the shortest measured execution time, while critical and atomic synchronization incurred higher costs because threads repeatedly accessed a shared counter.

The findings demonstrate the importance of selecting appropriate synchronization mechanisms to ensure correctness and improve parallel performance.

Task 4: False Sharing

17. Write a program in which T threads each increment their own entry of int counter[T] 10,000,000 times.

An OpenMP program was implemented in which each thread increments a separate element of a shared integer array 10,000,000 times.

```
#pragma omp parallel num_threads(T)
{
    int tid = omp_get_thread_num();
    for (int i = 0; i < 10000000; i++) {
        counter[tid]++;
    }
}
```

Although threads update separate counters, adjacent array elements may occupy the same cache line, potentially causing false sharing.

18. Measure runtime with T = 4 and T = 8.

The unpadded implementation was evaluated using four and eight threads, with five timed repetitions per configuration.

Threads	Mean execution time (seconds)
4	0.013249
8	0.015378

The unpadded implementation exhibited a higher mean execution time with eight threads, consistent with increased contention for shared cache-line resources.

19. Modify the code to pad each entry to a 64-byte cache line.

The program was modified to place each thread-specific counter in a separate 64-byte-aligned structure.

```
typedef struct {
    _Alignas(64) volatile int counter;
    unsigned char padding[64 - sizeof(int)];
} PaddedCounter;
```

This arrangement increases the memory separation between counters, reducing the likelihood of false sharing.

20. Measure runtime again.

The padded implementation was tested using the same iteration count, compiler settings, and thread configurations.

Threads	Mean execution time (seconds)
4	0.011449
8	0.010952

Padding reduced the measured execution time in both configurations, while all correctness checks passed.

21. Report the speedup from padding.

The padding speedup was calculated as:

$$S_{\text{padding}}(T) = \frac{T_{\text{unpadded}}(T)}{T_{\text{padded}}(T)}$$

For four threads:

$$S_{\text{padding}}(4) = \frac{0.0132492}{0.0114492} = 1.1572$$

For eight threads:

$$S_{\text{padding}}(8) = \frac{0.0153784}{0.0109518} = 1.4042$$

Dalya Mohammad Hassan
220253121

Table: Padding Performance Comparison

ß	Unpadded (s)	Padded (s)	Speedup
4	0.013249	0.011449	1.1572×
8	0.015378	0.010952	1.4042×

The results demonstrate that padding improved measured performance, with a larger benefit at eight threads.

22. Explain what false sharing is and why padding helps.

False sharing occurs when different threads modify independent variables located within the same cache line. Although the variables are logically unrelated, cache-coherence mechanisms may transfer or invalidate the shared cache line between processor cores, increasing memory-system overhead.

Padding separates thread-specific counters into distinct aligned memory regions, reducing unnecessary cache-line interference.

The experiment showed that padding improved execution time for both tested configurations. The observations are consistent with reduced false sharing, although cache-coherence traffic was not directly measured.

Task 4 Conclusion

The experiment demonstrated the performance impact associated with placing frequently updated thread-specific counters close together in memory.

Using 64-byte padding improved performance by factors of 1.1572 and 1.4042 for four and eight threads, respectively. All configurations passed correctness verification.

These results illustrate how memory layout can influence the performance of shared-memory parallel applications, even when threads operate on logically independent variables.

Task 5: Scaling Analysis and Report
23. For Task 1, plot speedup vs. threads and efficiency vs. threads.

Parallel speedup and efficiency were calculated using the mean serial execution time of 21.197242 seconds.

Speedup:

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)}$$

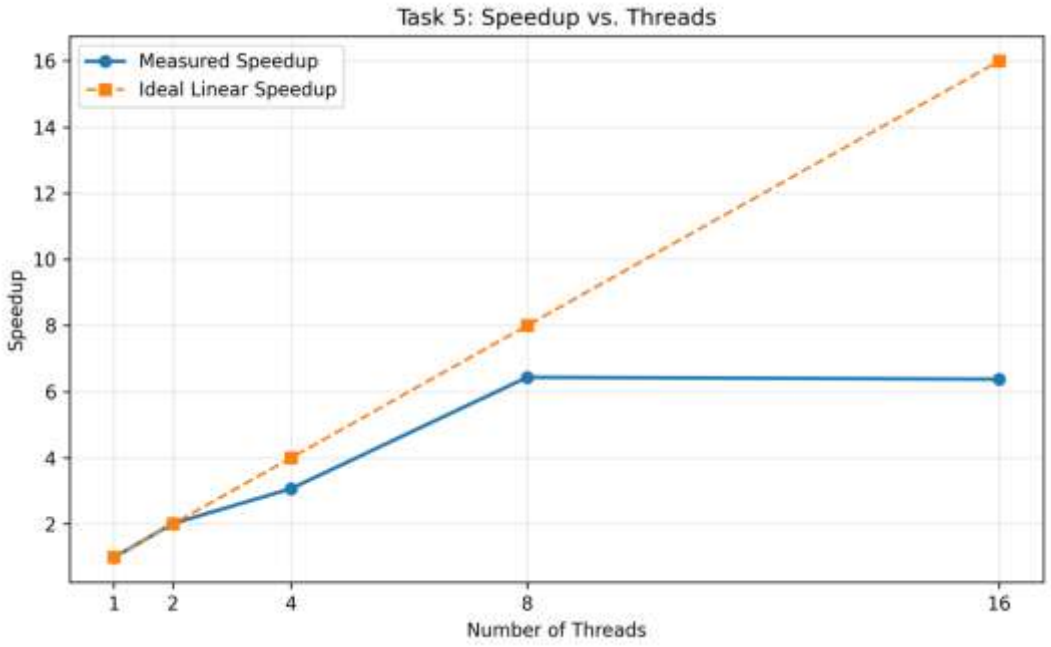
Parallel efficiency:

$$E(p) = \frac{S(p)}{p} \times 100\%$$

Table 10. Measured Parallel Scaling Results

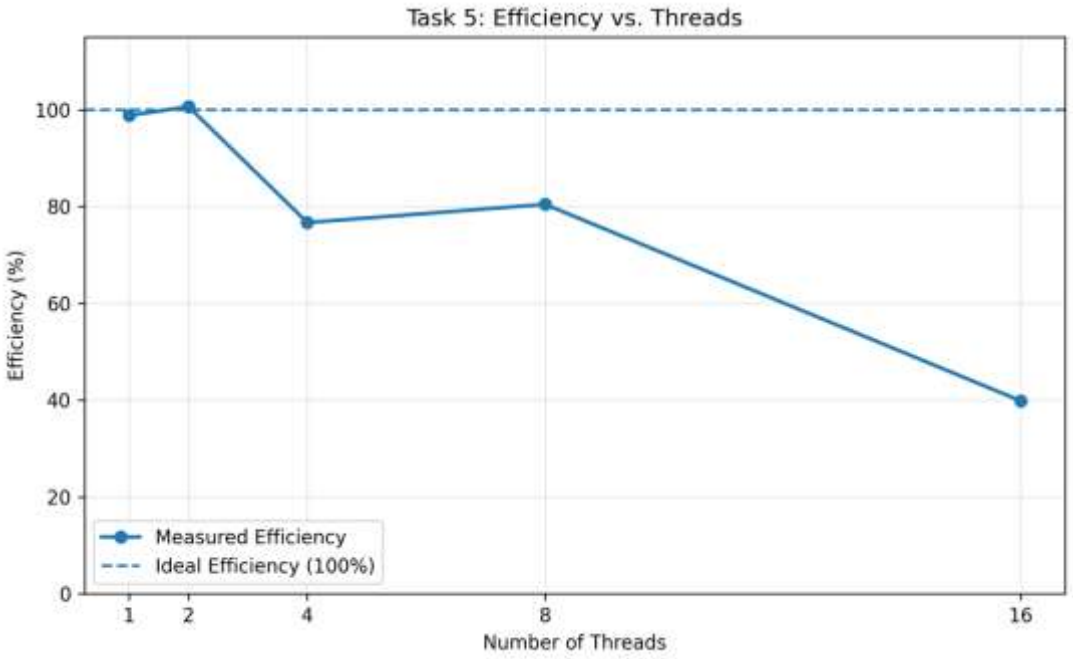
Threads	Mean runtime (s)	Speedup	Efficiency (%)
1	21.457594	0.9879	98.79
2	10.531808	2.0127	100.63
4	6.910633	3.0673	76.68
8	3.293501	6.4361	80.45
16	3.323797	6.3774	39.86

Figure 1. Measured speedup versus thread count, compared with ideal linear speedup.



Dalya Mohammad Hassan
220253121

Figure 2. Parallel efficiency versus thread count.



The highest measured speedup was 6.4361, achieved with eight threads and an efficiency of 80.45%.

24. Discuss observed deviations from linear scaling.

Ideal linear scaling produces a speedup equal to the number of threads. However, the measured performance deviated from ideal scaling.

At eight threads, the measured speedup was 6.4361, compared with the ideal speedup of 8. Increasing the thread count to 16 slightly decreased the speedup to 6.3774 and reduced efficiency to 39.86%.

The results suggest diminishing returns due to possible memory-system limitations, parallel overhead, and resource contention.

The Apple M4 Pro has 12 CPU cores, consisting of eight performance cores and four efficiency cores. Requesting 16 software threads may introduce additional scheduling contention.

The efficiency of 100.63% at two threads is slightly above ideal. This small deviation may reflect timing variability or cache-related effects and does not establish sustained superlinear performance.

25. List at least three bottlenecks.

1. Memory Access and Cache Locality

The matrix multiplication algorithm accesses matrix B noncontiguously in its innermost loop. This may increase cache misses and reduce memory-access efficiency. GCC optimization diagnostics also identified missed vectorization opportunities involving strided memory accesses.

2. Parallel Execution Overhead

OpenMP introduces overhead associated with thread management, work distribution, and coordination. Such overhead may reduce the achievable speedup compared with ideal linear scaling.

3. Processor Resource Contention

The Apple M4 Pro contains 12 CPU cores with heterogeneous processing capabilities. Running 16 OpenMP threads may cause resource contention and scheduling overhead, limiting further speedup.

These are potential bottlenecks consistent with the observed results; their individual effects were not directly measured.

26. Suggest two improvements you would try next.

Improvement 1: Cache-Aware Matrix Multiplication

Implement blocked (tiled) matrix multiplication to improve cache locality and data reuse. Processing smaller matrix blocks may reduce cache misses and improve computational performance.

Improvement 2: Thread Affinity and Scheduling Optimization

Investigate thread affinity, thread count, and OpenMP scheduling policies to reduce unnecessary thread migration and resource contention. This may improve execution efficiency on the heterogeneous Apple M4 Pro processor.

Both improvements should be evaluated experimentally under the same workload and compiler settings.

Task 5 Conclusion

The scaling analysis demonstrated that OpenMP improved matrix multiplication performance, achieving a maximum measured speedup of 6.4361 with eight threads and an efficiency of 80.45%.

Increasing the thread count to 16 did not improve mean runtime and reduced efficiency to 39.86%.

These results illustrate the limitations of strong scaling for a fixed-size workload and highlight the importance of memory locality, parallel overhead, and processor architecture.